

Exercitiul 2 — Observer: un eveniment, N reactii

Cerința exercițiului 2 din modulul de design patterns.

Context

Aceeași comandă din magazin, dar acum urmărim **starea** ei: **Plasata** -> **Expediata** -> **Livrata**. La FIECARE schimbare de stare, mai multe parti trebuie să reacționeze:

- clientul primește un email;
- se scrie o linie în jurnalul de livrări;
- depozitul își pregătește pasul următor.

Prima variantă la care se gândește oricine — și e din nou o capcană:

```
public void SchimbaStare(string stareNoua)
{
    Stare = stareNoua;
    email.Trimite(stareNoua);
    jurnal.Scrie(stareNoua);
    depozit.Pregateste(stareNoua);
}
```

Comanda ajunge să cunoască fiecare parte concretă pe nume. Când marketingul cere și o notificare push, RE-DESCHIZI **SchimbaStare** și mai adaugi o linie — iar **Comanda**, care ar trebui să se ocupe de comandă, știe acum despre email, SMS, jurnal, depozit, push... Fix problema de la ex1 (Strategy), dar pe alta axă: acolo se schimbă **CUM se calculează**, aici se schimbă **CINE reacționează**.

Ideea

Inversezi dependența. **Comanda** (numită **Subject**) nu mai cunoaște nicio parte concretă — ține o listă de **observatori** care au semnat un contract, și când se schimbă starea îi **anunță pe toți**, la fel, fără să știe cine sunt:

```
public interface IObservable
{
    void Actualizeaza(string stareNoua);
}
```

O parte noua care vrea sa reactioneze = o clasa noua care implementeaza `IObservable` si se aboneaza. `Comanda` ramane neatinsa. Asta e **Observer**: o singura sursa de eveniment, N ascultatori care reactioneaza independent.

Contractul si rolurile

Rol	In exercitiu	Ce face
Subject (sursa)	<code>Comanda</code>	tine observatorii, isi schimba starea, ii anunta pe toti
Observer (contractul)	<code>IObservable</code>	declara <code>Actualizeaza(string stareNoua)</code>
ConcreteObserver	<code>NotificatorEmail</code> , <code>JurnalLivrare</code> , <code>PanouDepozit</code>	fiecare reactioneaza in felul lui

Ce afiseaza fiecare observator

Observer	La <code>Actualizeaza("X")</code> afiseaza
<code>NotificatorEmail</code>	<code>[EMAIL] Comanda a trecut in starea: X</code>
<code>JurnalLivrare</code>	<code>[LOG] Stare inregistrata: X</code>
<code>PanouDepozit</code>	<code>[DEPOZIT] Pregatesc pentru: X</code>

Pasii (in ordinea asta — primul e capcana)

1. **Simte durerea intai.** Pe ciorna, scrie `SchimbaStare` cu apeluri directe (`email.Trimite(...)` , `jurnal.Scrie(...)`). Raspunde: cate locuri atingi cand adaugi o notificare push? Ce cunoaste `Comanda` si n-ar trebui?
2. **Scoate contractul.** Cele 3 clase implementeaza `IObserver` , fiecare cu afisarea ei.
3. **Fa `Comanda` sa anunte, nu sa stie.** `SchimbaStare(stareNoua)` seteaza `Stare` si apoi PARCURGE observatorii chemand `Actualizeaza(stareNoua)` — cu un `for` , fara sa cunoasca vreo clasa concreta.

Cerinte

1. Interfata `IObserver` + cele 3 observatoare.
2. `Comanda` (Subject) cu: `Stare` (proprietate citibila), un `IObserver[]` primit prin constructor, si `SchimbaStare(string stareNoua)` care seteaza starea si anunta toti observatorii.
3. `Testare2` (chemata din `Program.cs`): creezi cele 3 observatoare, o `Comanda` cu ele, apoi `SchimbaStare("Expediata")` si `SchimbaStare("Livrata")` . In output trebuie sa vezi, la fiecare schimbare, toate cele 3 reactii — deci 6 linii, grupate cate 3.

Constrangeri

- `Comanda` NU cunoaste nicio clasa concreta de observator — doar `IObserver` .
- Fara `is` , `as` , `GetType()` . Fara LINQ, `List` , `Dictionary` — doar `IObserver[]` si `for` .
- Mesajele de eroare in engleza.

Cum rulezi

```
dotnet run
```

Bonus

- Adauga `Aboneaza(IObserver nou)` si `Dezaboneaza(IObserver vechi)` pe `Comanda` , care cresc/micsoreaza array-ul manual (fara `List`). Aboneaza un observator DUPA prima schimbare de stare si observa ca prinde doar schimbarile urmatoare — un observator vede doar ce se intampla dupa ce s-a abonat.

- Intrebare de gandit, legatura cu ex1: la Strategy, **Comanda** avea UN singur colaborator (strategia) si-l INTREBA („cat costa?“). La Observer are MULTI si ii ANUNTA („s-a schimbat starea“). Care e diferenta de directie intre „a intreba“ si „a anunta“, si de ce Observer nu asteapta niciun raspuns inapoi?

